

CAPSTONE PROJECT REPORT

Job Postings Skill Demand

Databricks and Snowflake pipeline for synthetic technology skill-demand analysis

Student name	Aadi Ratn
Student ID	23051560
Batch	Databricks and Snowflake 2026
Submission date	25 September 2026
Project topic	Topic 26, Capstone Project Briefs, pages 104-107

Declaration

This report documents a pipeline built and tested on the supplied synthetic job-postings scope. Findings describe the generated dataset and do not claim to measure the live labour market.

Submitted to: Trainer / Faculty

1. Problem Statement

Technology teams need a clear view of which skills occur most often together and how their representation changes over time. Raw job-posting data is unsuitable for analysis because it contains duplicate records, invalid dates, missing skills and inconsistent skill spellings. This project builds a controlled analytics pipeline that transforms synthetic job postings into validated, queryable skill-demand datasets.

2. Objectives and Scope

Objective	Evidence of completion
Generate and ingest source data	55,350 synthetic postings across January 2024 to June 2025.
Clean and standardize records	51,840 valid postings retained; 57 skill variants mapped to 20 canonical skills.
Publish reusable metrics	360 monthly skill rows and 6,660 monthly pair rows exported.
Load and query in Snowflake	Three staged CSV files loaded; 14 of 14 full-result checks passed.

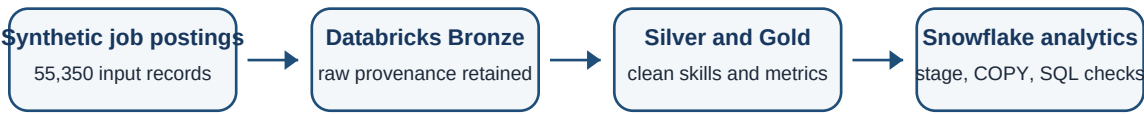
Table 1. Objectives are linked to observed execution evidence.

The scope is intentionally bounded. It analyzes generated postings only, uses an 18-month period, schedules the Databricks portion, and uses a tested manual CSV handoff to Snowflake. It does not scrape websites, forecast hiring or claim unattended cross-platform ingestion.

3. Solution and Features

- Bronze, Silver and Gold layers with preserved raw provenance and explicit rejection reasons.
- Monthly top-ten skill demand, quarter-on-quarter share changes and directional companion-skill analysis.
- Validation gates, deterministic exports and repeat-load protection in Snowflake.
- A static dashboard that presents the verified data product for assessors.

4. System Architecture



Validated end to end: local Spark, Databricks, cloud CSV reconciliation and Snowflake

The scheduled component is Databricks. The CSV transfer to Snowflake was tested manually.

Figure 1. The implemented data path and validation boundary.

Raw synthetic postings are transformed in Databricks. Bronze retains the source values and provenance. Silver removes duplicates and invalid rows before splitting and standardizing skills. Gold contains monthly skill measures and co-occurrence measures. Curated CSV exports are placed in a Snowflake named stage and loaded with COPY INTO before analytical SQL and full-result verification are run.

5. Technology Stack

Component	Purpose
Python and PySpark	Deterministic data generator, transformations and local verification.
Databricks serverless	Cloud execution, Delta Bronze/Silver/Gold tables and scheduled job.
Snowflake	Named-stage load, duplicate-load audit, analytical SQL and verification views.
CSV and SHA-256	Portable curated handoff and reproducibility fingerprints.
HTML, CSS and JavaScript	Hosted static dashboard using verified project metrics.

Table 2. Components used in the submitted implementation.

6. Data Design and Layers

Layer	Main content	Purpose
Bronze	Raw postings and companies	Preserve input values and source lineage.
Silver	Posting-skill records	Reject bad records; normalize and explode skills.
Gold	Skill-month and pair-month metrics	Supply ranked demand and companion-skill queries.

Table 3. Layered data design.

7. Implementation Highlights

The implementation eliminates duplicates before exploding the skill list, preventing duplicated postings from multiplying downstream measures. It splits comma- and semicolon-separated skill lists, trims whitespace, lowercases values and maps aliases to canonical names. Skill share uses accepted postings in the month as its denominator, so it is a comparable per-skill measure rather than a distribution that must total 100%.

Control	Implemented behavior
Input rejection	Duplicate, missing-skill, unknown-company and invalid-date records receive explicit reasons.
Determinism	A fixed generated scope, ordered results and SHA-256 result fingerprints make reruns comparable.
Databricks compatibility	The date offset is explicitly cast to INT for Spark 4; the original source is preserved.
Query correctness	Pair aggregation occurs in a CTE before ranking; quarterly division uses explicit DOUBLE precision.

Table 4. Implementation decisions that protect metric correctness.

8. Data Quality Results

Measure	Observed result
Input postings	55,350
Accepted postings	51,840
Duplicate records rejected	1,350
Missing-skill records rejected	1,350
Unknown-company records rejected	540
Invalid-date records rejected	270
Canonical skills	20 from 57 raw variants

Table 5. Observed data-quality outcomes from the completed pipeline.

Every valid posting is represented in Silver. The project stores rejected rows with raw values and a reason, allowing the pipeline to be audited without silently discarding input records.

9. Testing and End to End Results

Test area	Expected result	Observed result
Local Spark pipeline	Repeatable outputs and all validations	Two full runs; all 31 checks passed; identical exports.
Independent reconciliation	Gold and Silver agree with an oracle	All posting-skill, monthly, pair and rejection values agree.
Databricks pipeline	Tables and notebook expectations complete	All 26 expectations passed.
Scheduled execution	A scheduled cloud run completes	Run 365017355708433 succeeded in 2 minutes 42 seconds.
CSV transfer	Cloud exports match local evidence	Every downloaded row agrees; tolerance 1e-12 for floating measures.
Snowflake first load	All curated rows load without errors	360, 6,660 and 227,624 rows loaded; zero errors.
Repeated Snowflake load	Same files add no duplicate data	All three files LOAD_SKIPPED with zero rows added.
Snowflake analysis	Results match expected full outputs	14 checks passed; 0 failed. Q1 180, Q2 120 and Q3 60 rows.

Table 6. End-to-end evidence collected from the implementation.

10. Evaluation Metrics

The completion criterion is not only whether code runs. The submission uses row counts, rejection counts, deterministic fingerprints, full-row reconciliation, repeated-load behavior and analytical-query result sets. These checks demonstrate that the values shown in the dashboard can be reproduced from the same curated exports.

Metric	Verified value
Monthly Gold rows	360
Pair Gold rows	6,660
Silver posting-skill rows	227,624
Snowflake final checks	14 passed / 0 failed
Scheduled run duration	2 minutes 42 seconds

11. Results and Interpretation

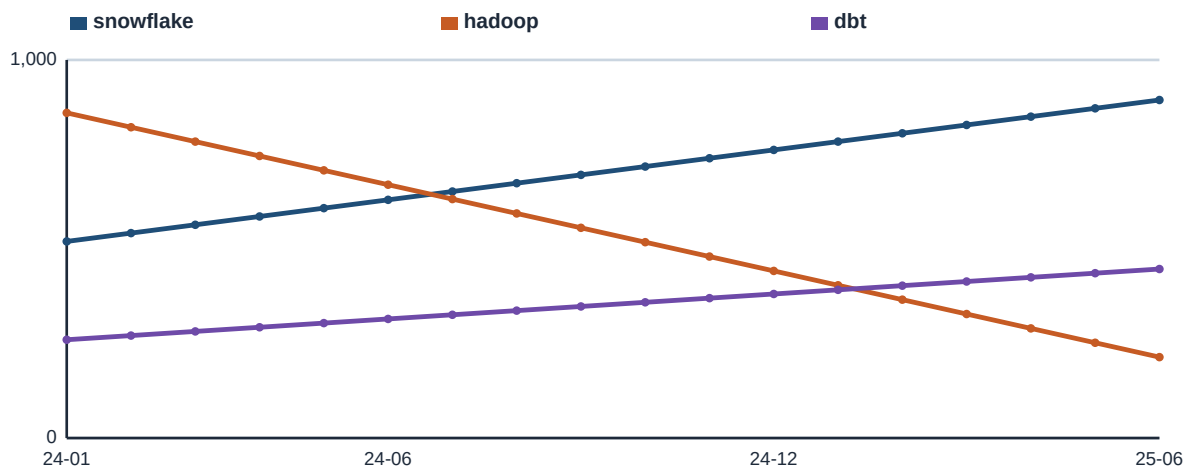


Figure 2. Verified monthly mentions from the project Gold dataset. Snowflake rises, Hadoop declines and dbt grows over the synthetic period.

The trend chart is descriptive of the generated data. Snowflake mentions increase from 520 in January 2024 to 894 in June 2025. Hadoop decreases from 860 to 214 over the same period. The co-occurrence analysis identifies dbt as a companion to Snowflake with a verified directional confidence of 50% in the final result set.

Rank	Skill	June 2025 postings	Share of 2,880 postings
1	scala	2,322	80.62%
2	java	1,133	39.34%
3	snowflake	894	31.04%
4	databricks	706	24.51%
5	docker	585	20.31%
6	gcp	585	20.31%
7	kubernetes	585	20.31%
8	terraform	585	20.31%
9	powerbi	571	19.83%
10	tableau	556	19.31%

Table 7. Top ten skills in the last project month, June 2025.

12. Dashboard and Submission Assets

The hosted dashboard is a lightweight static site built specifically for this capstone. It shows the same verified scope, architecture, data-quality measures and month-level trend values that are retained in the repository. It uses no confidential credentials and can be hosted on GitHub Pages, Netlify or Vercel.

Submission item	Prepared content
Project details	Final project name and brief description for the form.
Working-project folder	Source code, SQL, notebook, report, evidence and a verified project archive.
GitHub repository	A clean repository containing only the project and dashboard assets.
Hosted project	Responsive static dashboard, ready to deploy from the repository.
Report PDF	This evidence-based capstone report with run instructions and references.

Table 8. Submission deliverables prepared for the final form.

13. Challenges and Learnings

- Spark 4 requires an integer date offset. Preserving the original source alongside the compatible execution path kept the change auditable.
- The analytical queries needed engine-aware SQL: aggregate first, then rank; make decimal precision explicit for cross-engine comparison.
- A rerun-safe load must be tested, not assumed. The repeated COPY check confirms Snowflake did not load the same files twice.
- Live cloud results were reconciled against local outputs so an individual success message was not treated as proof of analytical correctness.

14. Future Improvements

- Add controlled snapshot replacement or file-version management for future ingestion cycles.
- Add an orchestration layer when unattended Databricks-to-Snowflake integration is required by a future scope.
- Use a real, ethically sourced dataset with documented permissions if the project later needs market-level conclusions.
- Add filters for city, sector and company size to extend the dashboard analysis.

15. Conclusion

Job Postings Skill Demand is feasible and complete within its stated scope. It converts synthetic raw job postings into validated analytical tables, exports a portable curated dataset, loads it into Snowflake and verifies both loading behavior and analytical results. The completed run evidence includes local Spark, Databricks interactive and scheduled execution, cloud-export reconciliation, Snowflake COPY validation and full-query checks.

The project is ready to be finalized as Topic 26 because it produces a working, reproducible pipeline and an assessable dashboard without overstating the data source or the level of automation.

16. Reproduction Steps

Step	Action
1	Run job_postings_skill_demand/verification/run_local.py with PySpark and DuckDB.
2	Import the Databricks notebook and run all cells on serverless.
3	Download the three exported CSV files from the Databricks Volume.
4	Run Snowflake setup/load SQL, upload CSVs to the named stage and execute COPY INTO.
5	Run analysis SQL and 03_full_result_verification.sql to reproduce the result checks.
6	Open the dashboard folder through a static host to present the published evidence.

Table 9. High-level path to reproduce the demonstrated implementation.

17. References and Project Links

- Capstone Project Briefs, Topic 26, pages 104-107 (provided course document).
- Databricks documentation: input_file_name and file-provenance guidance.
- Snowflake documentation: COPY INTO table and data-load considerations.
- Project repository README, FINAL_VERIFICATION.md and Snowflake verification SQL included in the submission folder.

Prepared from verified execution evidence dated 10 September 2026.